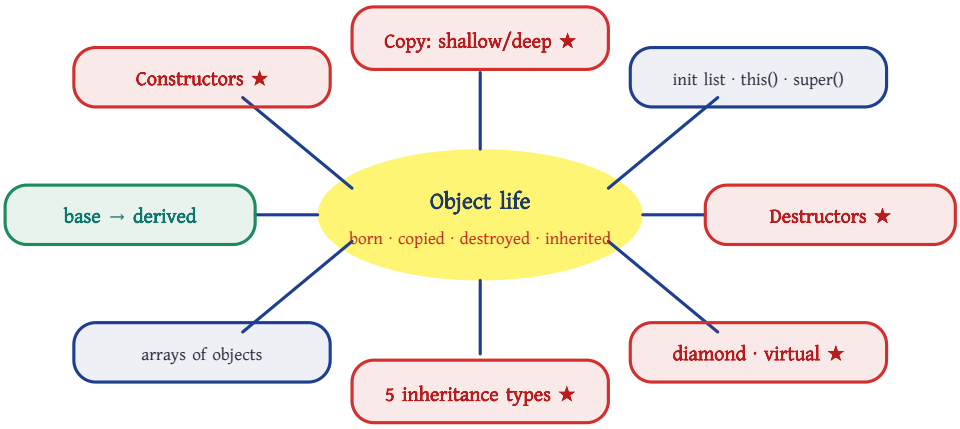
Object Life & Inheritance

Constructors · Copy · Destructors · Arrays · Pointers · Inheritance: exam-ready in 12 sheets



↑ the whole lesson on one page: red = high-yield, green = the rule everything else hangs on

Syllabus coverage

The OOP syllabus continues: "**Constructors, destructors and copy constructors; Array of objects, object pointers, and object references; Inheritance: single and multiple inheritance**". Classes, access specifiers and `static` were the previous lesson; virtual functions, overriding and abstract classes are the next.

High-yield list

① Ctor: class name, no return type
② Free default ctor only if none written
③ Copy ctor: `A(const A&)`
④ Default copy = shallow
⑤ `~A()`: no args, no overload
⑥ Build base → derived; destroy reverse
⑦ Java `new A[3]` → 3 nulls
⑧ `new` calls ctor; `malloc` does not
⑨ Java: 1 extends, many implements
⑩ Diamond → virtual base class

Legend: **yellow** = key term · **red circle** = must-memorise · **purple box** = memory trick · **red box** = exam trap · **green** = worked example

Contents

✓ Constructors: rules & types … p. 2
✓ Copy constructor; shallow vs deep … p. 3
✓ Initializer list & chaining … p. 4
✓ Destructors & object lifetime … p. 5
✓ Arrays of objects … p. 6
✓ Pointers, `this`, new/delete … p. 7
✓ Inheritance: is-a & five types … p. 8
✓ Access modes & build order … p. 9
✓ Diamond problem; Java interfaces … p. 10
✓ Night-before sheet … p. 11
✓ Practice MCQs + written … p. 12

1 Constructors: Rules and Types

Definition: A **constructor** is a special member function that runs **automatically** when an object is created and puts it in a valid initial state. It has the **same name as the class** and **no return type — not even `void`**. It can be overloaded, can be `private` (singleton), but cannot be `virtual` or `static` in C++, and cannot be `final`, `static` or `abstract` in Java. It is **not inherited**.



constructor = first call of an object's life; destructor = last

the life of every object in five steps

```
class Student {          // C++
    string name;  int age;
public:
    Student() {          // default
        name = "Unknown"; age = 0;
    }
    Student(string n, int a) {  // param.
        name = n; age = a;
    }
    Student(const Student& s) { // copy
        name = s.name; age = s.age;
    }
};
Student s1;              // default
Student s2("Alice", 20); // parameterized
Student s3 = s2;         // copy
```

Type	Signature	Runs for
Default	`A()`	`A a;`
Parameterized	`A(int x)`	`A a(5);`
Copy	`A(const A&)`	`A b = a;`
Implicit	compiler's	if you write **none**
Overloaded	many lists	picked at compile time

Rule	C++	Java
Free default ctor	only if **no** ctor declared	only if none written; calls `super()`
Copy constructor	implicit, member-wise (shallow)	**none**: write one or use `clone()`
Add a return type	compile error	`void A(){}` = a plain **method**
Object without `new`	`A a;` creates it (stack)	`A a;` is only a null reference

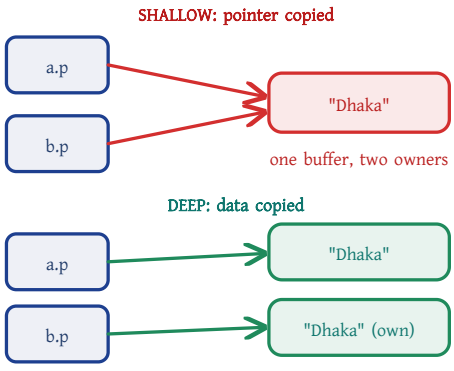
Memory trick: "**A constructor is a birth certificate**: it carries the baby's name, is issued once at birth, and returns nothing."

Exam trap: Write only `A(int)` and `A a; / new A()` **stop compiling**. C++ `A a();` declares a **function**, not an object.

Worked example ✍️ which one runs?:
`A a; A b(3); A c = b; A d = 7;` → default, parameterized, **copy**, parameterized (implicit conversion, blocked by `explicit`).

2 Copy Constructor; Shallow vs Deep Copy

Definition: A **copy constructor** creates a **new** object as a copy of an existing object of the same class. C++ form: `ClassName(const ClassName& other)`. It runs when an object is **initialised** from another (`A b = a;`, `A b(a);`), when an object is **passed by value**, and when one is **returned by value** (often elided). The compiler's version copies member by member — a **shallow copy**.



shared vs own buffer

```
class Str {
    char* p;
public:
    Str(const char* s) {
        p = new char[strlen(s) + 1];
        strcpy(p, s);
    }
    Str(const Str& o) {     // deep copy
        p = new char[strlen(o.p) + 1];
        strcpy(p, o.p);
    }
    ~Str() { delete[] p; }
};
Str a("Dhaka");
Str b = a;   // copy ctor: own buffer
```

Point	Shallow copy	Deep copy
Copies	the pointer (address)	the data the pointer points to
Made by	compiler's copy ctor / `Object.clone()`	your own copy ctor / overridden `clone()`
After copy	change via *b* shows in *a*	objects independent
Danger	**double delete**, dangling pointer	slower, more memory
Fine when	no pointer/handle members	class owns heap memory, file, socket

Why `const A&`?: **By reference**: a by-value parameter would need a copy to make the copy → endless recursion, so C++ rejects `A(A a)`. **const**: lets you copy `const` objects and temporaries, and promises not to change the source. **Rule of three**: a class that needs its own destructor also needs its own copy constructor and copy assignment.

Java has no copy constructor by default: Write one yourself: `Student(Student s) { name = s.name; age = s.age; }`, or override `clone()`. `Object.clone()` is **shallow**, is `protected`, and throws `CloneNotSupportedException` unless the class `implements Cloneable`.

Memory trick: "**Shallow copies the address; deep copies the house.**" Two people with one house key fight over the furniture.

Exam trap: `b = a;` when *b* already exists calls the **assignment operator**, not the copy constructor. `A b = a;` is initialisation → copy constructor.

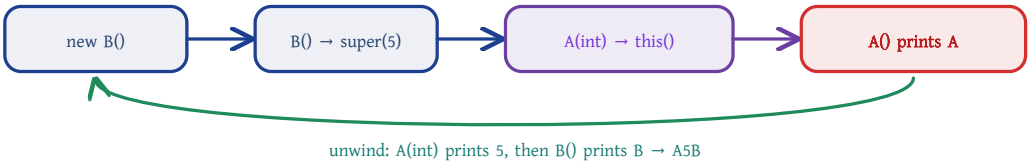
Worked example ✍ count copies: `void f(A x){}; A a; A b = a; A c(a); f(a); b = c;` → copy ctor: **3** calls; last is assignment.

3 Initializer List and Constructor Chaining

Definition: C++'s **member initializer list** (`A(int v) : x(v), Base(v) {}`) initialises bases and members **before** the body runs. It is **mandatory** for `const` members, reference members, members whose class has no default constructor, and a base class that needs arguments. **Constructor chaining** is one constructor calling another: Java `this(…)` (same class) and `super(…)` (parent), C++ `: Base(…)` and, since C++11, delegating `A() : A(0) {}`.

```
class Point {                // C++
    const int x;  int& r;
public:
    Point(int a, int& b)
        : x(a), r(b) {}  // must
};
class P3 : public Point {
    int z;
public:
    P3(int a, int& b, int c)
        : Point(a, b), z(c) {}
    P3(int& b) : P3(0, b, 0) {}
};          // C++11 delegating ctor
```

```
class A {                    // Java
    A() { System.out.print("A"); }
    A(int x) {
        this();          // 1st line
        System.out.print(x);
    }
}
class B extends A {
    B() {
        super(5);        // 1st line
        System.out.print("B");
    }
}
new B();   // prints A5B
```



unwind: A(int) prints 5, then B() prints B → A5B

calls go up the chain first; bodies print on the way back

Job	C++	Java
Call parent ctor	`: Base(args)` in the list	`super(args)`, **first line**
Call own other ctor	`: A(args)` (C++11)	`this(args)`, first line
Write neither	base default ctor runs	compiler adds `super();`
Both together?	delegating: nothing else	**no**, only one
Init order	bases, then members in **declaration order**, not list order	

Memory trick: "**Parents are built before children**": `super` goes first, the body last. Calls travel up, prints come back down.

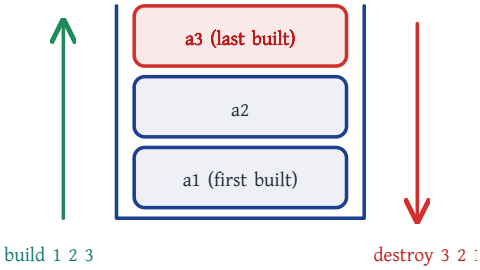
Exam trap: Parent has only `A(int);` `class B extends A { B(){} }` → **compile error**: the hidden `super()` finds no `A()`. Fix: `super(0);`.

Worked example ✍ list order: `int a, b;` then `T() : b(1), a(b) {}` → *a* is set **first**, from unset *b*: **garbage**.

4 Destructors and Object Lifetime

Definition: A **destructor** (`~ClassName()`) runs automatically when an object's lifetime ends, to release what it holds — heap memory, files, sockets, locks. Rules: tilde + class name; **no parameters, no return type**; therefore **cannot be overloaded** — exactly one per class; not inherited; may be `virtual` (a constructor never can). Objects are destroyed in the **reverse order** of construction.

Object kind	Destructor runs
local (automatic)	at the closing } of its scope, reverse order
global / `static`	at program exit (after `main` returns)
heap (`new`)	only on `delete` — forget it = **memory leak**
temporary	end of the full expression
member object	after the owner's destructor body
array element	highest index first



locals live on a stack: LIFO

Point	Constructor	Destructor
Name	`A()`	`~A()`
Parameters / overloading	any / yes	**none / no**
How many per class	many	**one**
Can be `virtual`	no	yes — needed in a polymorphic base
Order in inheritance	base → derived	derived → base

Virtual destructor (C++): `Base* p = new Derived; delete p;` — if `~Base()` is not `virtual`, only `~Base` runs (undefined behaviour; the Derived part leaks). Declare `virtual ~Base()` in any class meant to be a base. How virtual dispatch works: next lesson.

Java: no destructors: The **garbage collector** frees unreachable objects at a time you cannot predict; `System.gc()` is only a request. `finalize()` ran at most once, maybe never, and is **deprecated** (Java 9; for removal since 18). Close files with `try`-with-resources (`AutoCloseable`).

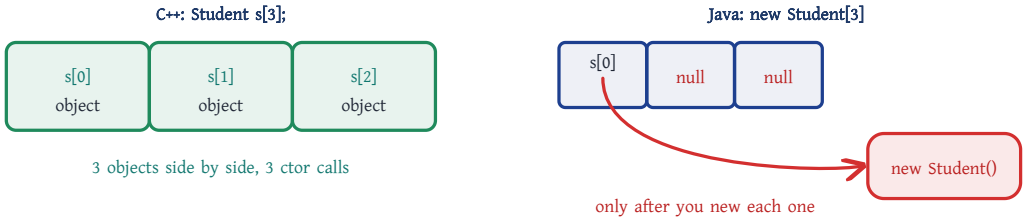
Memory trick: "**Plates on a stack**: the last plate put on is the first taken off." Tilde `~` = the wave goodbye.

Exam trap: Garbage collection is **not** a destructor: it reclaims memory only, at no fixed time. `finalize()` is not guaranteed to run.

Worked example ✍ trace: `{ A a(1); A b(2); A* p = new A(3); }` → built 1 2 3; at `}`: **~2 ~1**; object 3 is never destroyed (leak).

5 Arrays of Objects

Definition: An **array of objects** is an array whose elements are objects of a class. In **C++**, `Student s[3];` creates **three objects** and calls the default constructor three times, so the class needs an accessible default constructor (or every element must be initialised). In **Java**, `new Student[3]` creates one array of **three null references** — no Student object, no constructor call — until you `new` each element.



left: real objects · right: an array of references

```
// C++
Student s[3];              // 3 x Student()
Student t[2] = { Student("A", 20),
                 Student("B", 22) };
Student* p = new Student[n];
p[0].display();
delete[] p;    // [] required
```

```
// Java
Student[] s = new Student[3]; // nulls
s[0].display();     // NullPointerException
for (int i = 0; i < s.length; i++)
    s[i] = new Student();
s[0].name = "Alice";
s[0].display();     // now fine
```

Point	C++ `A arr[3];`	Java `A[] arr = new A[3];`
Creates	3 objects	1 array object + 3 references
Constructor calls	3 (default ctor)	0
Initial elements	constructed objects	`null` (numbers 0, boolean false)
No default ctor in class	compile error unless all initialised	no problem
Freed	scope end, or `delete[]` for `new A[n]`	garbage collector
Length	not stored (`sizeof` trick for fixed arrays)	`arr.length` (a field, no brackets)

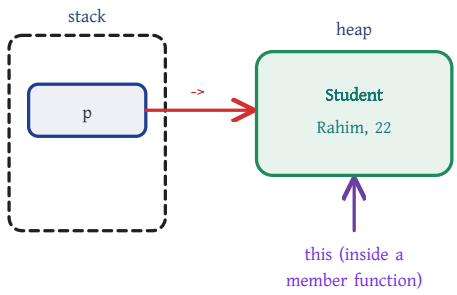
Memory trick: "C++ builds a row of houses; Java prints a row of empty address slips." No house until you `new` one.

Exam trap: `new A[n]` must be freed with `delete[]`; plain `delete` is undefined behaviour. C++ `A* q[3];` is an array of pointers: **no** objects.

Worked example ✍ output: `A(){cout<<"A";} ~A(){cout<<"~";}` and `{ A arr[3]; }` → **AAA~~~**. Java `new A[3]` with the same print: nothing.

6 Object Pointers, `this`, new/delete, References

Definition: An **object pointer** holds an object's address: `Student* p = &s;`. Members are reached with the arrow: `p->display()` ≡ `(*p).display()`. `new` creates an object on the heap and returns its address; `delete` destroys it. Every non-static member function gets a hidden pointer **`this`** to the object it was called on. Java has no explicit pointers: every object variable is a **reference**.



p on the stack, object on the heap

```
class Counter {
    int n = 0;
public:
    Counter& inc() {
        ++this->n;        // this->n == n
        return *this;     // allows chaining
    }
    int get() const { return n; }
};
Counter* c = new Counter;
c->inc().inc().inc();
cout << c->get();  // 3
delete c;  c = nullptr;
```

Point	`new` / `delete` (C++)	`malloc` / `free` (C)
What it is	operator (can be overloaded)	library function
Constructor / destructor	**called**	not called
Size and type	computed; returns `A*`	bytes by hand; returns `void*`
On failure	throws `std::bad_alloc`	returns `NULL`
Arrays	`new A[n]` … `delete[]`	`malloc(n*sizeof(A))` … `free`

Point	C++ pointer `A* p`	C++ reference `A& r = a`	Java reference
Can be null	yes	no; must be initialised	yes (`null`)
Re-point later	yes	no (an alias)	yes
Arithmetic / access	`p++`; `->` and *	none; `.`	none; `.`
Freeing the object	`delete` by hand	owner's lifetime	garbage collector

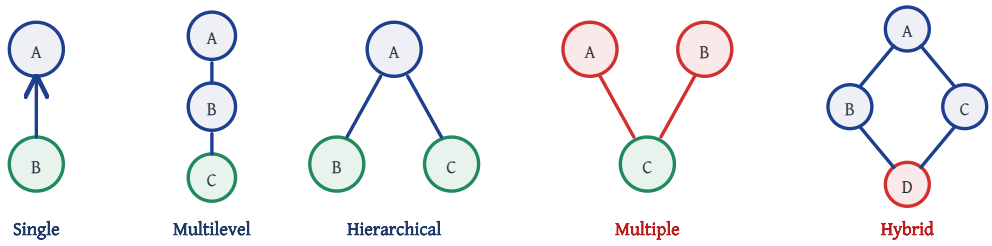
Memory trick: "**Arrow for an address, dot for the thing itself.**" `this` = "the object that answered the call".

Exam trap: `this` does not exist in `static` or `friend` functions. You cannot assign to `this`. Mixing `new` with `free` (or `malloc` with `delete`) is undefined.

Worked example ✍ Java alias: `Student s = new Student(); s.age = 20; Student r = s; r.age = 21;` → `s.age` is **21**: one object, two references.

7 Inheritance: is-a and the Five Types

Definition: **Inheritance** lets a new **derived** (child, sub) class acquire the members of an existing **base** (parent, super) class, then add or redefine its own. It models an **is-a** relationship (Car is-a Vehicle) and gives code reuse and the basis for run-time polymorphism. Syntax: C++ `class Car : public Vehicle {…};` · Java `class Car extends Vehicle {…}`.



children at the bottom; red = not allowed with Java classes

Type	Shape	C++ example	C++ / Java classes
Single	A → B	`class B : public A`	✓ / ✓ `extends`
Multilevel	chain A → B → C	`B : A`, `C : B`	✓ / ✓
Hierarchical	A → B, A → C	`B : A`, `C : A`	✓ / ✓
Multiple	A, B → C	`C : public A, public B`	✓ / ✗ (interfaces ✓)
Hybrid	mix, e.g. diamond	A; B, C : A; D : B, C	✓ / interfaces only

Not inherited: **Constructors** and the **destructor** (C++11 can opt in with `using Base::Base;`) · the copy **assignment operator** (hidden by the derived one) · **friend** functions — friendship is not inherited · **private** members: in C++ they sit inside the derived object but cannot be named; Java says they are not inherited. A `final` class (e.g. `String`) cannot be extended.

Point	is-a	has-a
Mechanism	inheritance	composition
Example	Car is-a Vehicle	Car has-a Engine
Coupling	tight	loose
Code	`extends`	a field

Memory trick: Five types: "**S**ome **M**ice **M**ake **H**uge **H**oles" = Single, Multilevel, Multiple, Hierarchical, Hybrid.

Exam trap: **Multilevel ≠ multiple.** Multilevel = a chain of single steps (Java allows it); multiple = two parents at once (Java classes do not).

Worked example ✍ classify: `C : A, B` → multiple. `B : A; C : B` → **multilevel**. `B : A; C : A` → hierarchical.

8 Inheritance Access Modes and Build Order

Definition: In C++ the **inheritance mode** — `public`, `protected` or `private` — is a **ceiling**: an inherited member can become less visible, never more. Base `private` members are **never accessible** in the derived class, whatever the mode. Default mode: `class` → private, `struct` → public. Java has one mode only: `extends` keeps every access level as it is.

In base ↓	`public` inheritance	`protected` inheritance	`private` inheritance
`public`	**public**	protected	private
`protected`	protected	protected	private
`private`	**not accessible**	**not accessible**	**not accessible**

```
class X { public:
    X()  { cout << "X"; }
    ~X() { cout << "~X"; } };
class Y { public:
    Y()  { cout << "Y"; }
    ~Y() { cout << "~Y"; } };
class Z : public X {
    Y y;                    // member
public:
    Z()  { cout << "Z"; }
    ~Z() { cout << "~Z"; } };
int main() { Z z; } // XYZ~Z~Y~X
```



destroy = build read backwards

1. **Virtual base classes** first (once each), then **direct bases** in the order of the base list — `class C : public B, public A` builds B before A, whatever the initializer list says.
2. **Member objects** in declaration order, then the **constructor body**. Destruction runs the exact reverse.
3. **Java:** static blocks once at class load (parent first); per `new`: the `super()` chain up to `Object`, then field initialisers and body, parent first.

```
class A { int a;            // private
  protected: int b;  public: int c; };
class B : private A {
  void f() { a = 1;   // ✗ base-private
             b = 2;   // ✓ now private in B
             c = 3; } };  // ✓ private in B
class C : public B {
  void g() { c = 4; } };  // ✗ private in B
B obj;  obj.c = 5;    // ✗ in main
```

Worked example ✍ read the table: `B : private A` makes A's *b* and *c* **private in B**: B's methods may use them; B's child *C* and `main` may not; A's *a* is unreachable even in B. With `B : public A`, `obj.c` and `c = 4` compile; *a* is **still locked**.

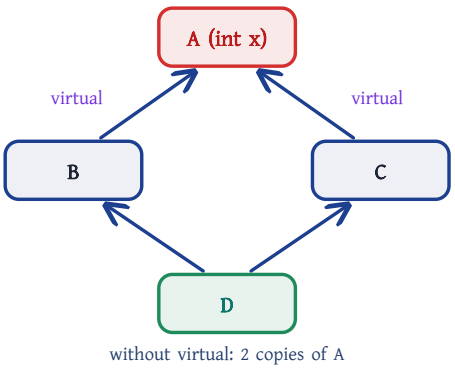
Memory trick: "**The mode is a ceiling.**" public keeps, protected lowers public to protected, private lowers everything to private; base-private stays locked.

Exam trap: `class B : protected A`, *A* has public `f()`: `B b; b.f();` in `main` → **compile error**; inside B's own methods `f()` is fine.

Worked example ✍ output: `A(){cout<<"A";} ~A(){cout<<"a";}`, same for *B* with "B"/"b", `class B : public A, B obj;` → **ABba**.

9 Diamond Problem and Virtual Base Class

Definition: The **diamond problem**: *B* and *C* both inherit *A*, and *D* inherits *B* and *C*. A plain *D* object then holds **two copies** of *A*, so `d.x` is **ambiguous** (compile error). C++ fixes it with a **virtual base class**: `class B : virtual public A` and `class C : virtual public A` share **one A**. Java avoids it for state by allowing only one parent class.



the diamond: one grandparent, two paths

```
class A { public: int x;
    A() { cout << "A"; } };
class B : virtual public A {
public: B() { cout << "B"; } };
class C : virtual public A {
public: C() { cout << "C"; } };
class D : public B, public C {
public: D() { cout << "D"; } };

D d;          // prints ABCD
d.x = 5;      // OK: one shared A
// without virtual: ABACD, and
// d.x is ambiguous (use d.B::x)
```

Point	Plain multiple inheritance	With `virtual` base
Copies of A inside D	two (one via B, one via C)	**one**, shared
`d.x`	ambiguous → `d.B::x` or `d.C::x`	fine
Who constructs A	B and C, each its own A	the most-derived class D, **first**
Constructor output	A B A C D	A B C D

```
interface Fly  { default void go()
                   { System.out.print("fly"); } }
interface Swim { default void go()
                   { System.out.print("swim"); } }
class Duck implements Fly, Swim {
    public void go() {        // must override
        Fly.super.go();       // pick one
    }
}
```

Java: interfaces instead: One `extends`, many `implements`: `class Car extends Vehicle implements Horn, Gps`. Since Java 8 interfaces may carry `default` methods, so a method-level diamond is possible: two defaults with the same signature → **compile error** until the class overrides, optionally calling `Fly.super.go()`. A class method beats an interface default.

Memory trick: "**Virtual base = one shared grandparent.**" Two parents, one grandmother at the family dinner, not two.

Exam trap: `virtual` goes on the **middle** classes (B and C), not on D. A **virtual base class** is not a **virtual function**; the latter is next lesson.

Worked example ✍ size view:
With 4-byte `int x`, a plain diamond *D* carries **2** copies of x; the virtual version carries 1 (plus hidden base pointers).

Night-Before-the-Exam Sheet

Fact sheet:

Constructor: class name, no return type

Default ctor supplied only if none written

Default copy = shallow, member-wise

Build: virtual bases → bases → members → body

Java `super()/this()`: first statement

`new` → ctor + `bad_alloc`; `malloc` → NULL

`p->f()` ≡ `(*p).f()`

Destructor: `~A()`, no args, one per class

Copy ctor: `A(const A&)`

Rule of three: dtor + copy ctor + operator=

Destroy: exact reverse

Java: no destructor; GC; finalize deprecated

`new[]` pairs with `delete[]`

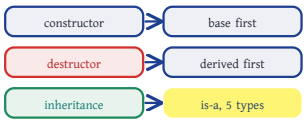
No `this` in static / friend functions

Top 12 one-liners for MCQs

1. Constructors can be overloaded; destructors cannot
2. Constructor cannot be virtual; destructor can
3. Constructors are not inherited
4. `A b = a;` → copy ctor; `b = a;` → operator=
5. C++ `A arr[3]` → 3 default-ctor calls
6. Java `new A[3]` → 0 ctor calls, 3 nulls
7. Inheritance = is-a; composition = has-a
8. Multilevel = chain; multiple = two parents
9. Java: multiple inheritance via interfaces only
10. Base private: never accessible in derived
11. C++ `class` inherits privately by default
12. Diamond → `virtual` base class

Top traps: `A a();` is a function · writing `A(int)` removes the free default ctor · copy ctor by value is illegal · GC ≠ destructor · `delete` on `new[]` is UB · init-list order = declaration order · `virtual` on B and C, not D.

All tricks together: birth certificate · shallow copies the address, deep the house · parents before children · plates on a stack · houses vs address slips · arrow for address, dot for thing · Some Mice Make Huge Holes · mode is a ceiling · one shared grandparent.



three ideas, three lines

Worked example ✍ one keyword changes the output: In the diamond `B : virtual public A`, `C : virtual public A`, `D : public B, public C` prints "**ABCD**". Drop both `virtual`s and the same `D d;` prints "ABACD" — and `d.x` stops compiling.

✓ Practice MCQs (exam pattern, 10 marks)

1. The return type of a constructor is:
   (a) void (b) int
   (c) none (d) the class type

2. Which is a valid C++ copy constructor of class A?
   (a) `A(A a)` (b) `A(const A& a)`
   (c) `A(A* a)` (d) `void A(A& a)`

3. A C++ destructor can:
(a) be overloaded (b) take arguments
(c) return int (d) be virtual

4. Java `Student[] s = new Student[3];` creates:
(a) 3 null references, no Student (b) 3 Student objects
(c) a compile error (d) 3 objects with default values

5. A prints "A"/"a" in ctor/dtor, B likewise "B"/"b"; `class B : public A; B obj;` prints:
(a) ABab (b) ABba
(c) BAba (d) BAab

6. C inherits B, and B inherits A. This is:
(a) multiple (b) hierarchical
(c) multilevel (d) hybrid

7. The diamond problem in C++ is solved by:
(a) a friend function (b) a virtual function
(c) a static member (d) a virtual base class

8. Under `protected` inheritance a base `public` member becomes:
(a) protected (b) public
(c) private (d) inaccessible

9. Which is true of `new` but not `malloc`?
(a) returns `void*` (b) needs size in bytes
(c) calls the constructor (d) returns NULL on failure

10. Which Java keyword calls the parent-class constructor?
(a) this (b) super
(c) extends (d) parent

Answer key → 1-c · 2-b · 3-d · 4-a · 5-b · 6-c · 7-d · 8-a · 9-c · 10-b

Written question: What is a copy constructor? When is it called? Explain shallow and deep copy with a C++ class that owns heap memory, and state the order of constructor and destructor calls in multilevel inheritance.
10 marks

1. **Definition (2)** — a constructor `A(const A& o)` that builds a new object from an existing one; by reference (by value would recurse), `const` to accept const objects and temporaries. 2
2. **When called (2)** — `A b = a;` / `A b(a);`, pass by value, return by value; not for `b = a;` (assignment operator). 2
3. **Shallow vs deep (4)** — default copy copies the pointer → shared buffer, double delete; deep copy allocates a new buffer and copies the data (the *Str* class with `new char[]`, `strcpy`, `delete[]`); rule of three. 4
4. **Order (2)** — for A → B → C: constructors A, B, C; destructors ~C, ~B, ~A. 2

Before the exam ✓

- ✓ Can list constructor and destructor rules
- ✓ Can write a deep-copy constructor
- ✓ Can trace build/destroy order
- ✓ Can fill the access-mode table
- ✓ Can draw the five inheritance types
- ✓ Can fix a diamond with `virtual`